CODE SPORT

Sandya Mannarswamy

In this month's column, we focus on virtual machines for languages. We will also discuss static and dynamic languages, and how the latter can be supported over virtual machines.

Over the last couple of columns, we have been looking at concurrency support in C++11 standard. However, this month, we take a break from that topic. Since virtualisation is *LFY*'s theme this month, let's focus on virtual machines for languages, why it makes sense to have a language VM, the static and dynamic languages, the different typing capabilities of the languages, and how these languages are implemented over virtual machines.

Language VMs

We are all familiar with the C language. We know that a program written in a high level language is compiled by our favourite C compiler on our machine to the native machine code. For instance, if you are compiling a C program on an X86 machine, your C compiler, 'gcc' for instance, translates your high level C code to X86 assembly instructions. Now, if you want to run the same program on, say, an IBM PowerPC, you can't just take the executable from your X86 machine and run it directly on the IBM PowerPC machine. (Of course, you can use binary emulators/ translators to do just this, but for now, we will assume that there are no binary emulators/ translators from X86 to PowerPC.) What you need to do is to take your C program and compile it to target PowerPC architecture so that the resulting executable can run on the PowerPC machine. Of course, this requires that you have a C compiler that can generate code for PowerPC.

The drawback with languages like C, which are natively compiled to machine code, is the lack of portability. If you want to run your program on a different architecture, you need to recompile. They are not 'Write Once, Run Anywhere'. On the other hand, consider a Java application like Helloworld.Java. Now, once you use a *javac* compiler to create the class file corresponding to your application, say 'Helloworld.class', you can virtually take it anywhere and run it on any platform on which Java is supported. The reason that your application, consisting of Java byte codes, is portable is because these byte codes do not actually run on the hardware platform directly. They run on an abstract machine called Java Virtual Machine (JVM) which abstracts away the details of the underlying hardware. As long as you have a JVM implementation on the target hardware machine on which you want to run your application, you can run your Java class files directly there. A language like C, in which you do native compilation of your application into the target machine's binary code, trades off the lack of portability with the additional performance it gains by compiling the application statically for the native platform.